

서울대학교 데이터사이언스대학원
Seoul National University · Graduate School of Data Science

Programming Practice for Data Science

26-FALL

QE Practice: C

Week 06 · 3 problems · 300 points · Suggested time 3 hours

Name _____

Student ID _____

Start Time _____

End Time _____

Instructions

This sheet has the same format as the C problem of the QE coding test. Treat it as an exam: set a timer, keep this sheet and a terminal open, and nothing else.

1. There are **three problems**, all in C, worth 100 points each. The points for each sub-part are shown next to it. Later sub-parts use the functions you wrote in earlier ones.
2. Each problem folder contains a header, `solution.c` and `main.c`. **Write your code only in `solution.c`.** Do not modify the header and do not rename any file. You may add helper functions and `#include` lines to `solution.c`.
3. Test your code with the given `main.c` and compare every line with the `Expected:` line printed under it:

```
cc -Wall -Wextra -o main main.c solution.c && ./main
```

   You may edit `main.c` to add your own checks. It is not graded.

4. **Print nothing except in the print function** of each problem (`deque_print`, `recent_print`, `graph_print`). Leftover debugging output is treated as a wrong answer.
5. **Free every block you allocate.** The free functions must release everything, and any buffer a function allocates for its own use must be freed before it returns. The grader counts calls to `malloc` and `free`.
6. Output formats are checked character by character: spaces, commas and newlines included.
7. You may look up the C standard library (for example on cppreference.com). Do not use any other material.

When the time is up, stop and grade your work. Test names start with the sub-part they check (`test_a_...`, `test_b_...`), so you can count your points per sub-part.

```
uv run runner.py test 6
```

Problem 1. Circular Deque and Round-Robin Scheduler (C)

Folder: `exercises/week_06_qe_c/c/impl_circular_deque/` **[100 points]**
Files: `deque.h` (do not modify) `solution.c` (write here and submit) `main.c` (self-check driver)

Implement a double-ended queue (deque) on a fixed-size array, then use it to simulate a round-robin CPU scheduler. Functions that can fail return `1` on success and `0` on failure.

```
typedef struct {
    int *buf;       /* array with capacity slots */
    int capacity;   /* maximum number of elements */
    int size;       /* current number of elements */
    int front;      /* index of the first element */
} Deque;
```

The elements, from front to back, are

```
buf[front], buf[(front + 1) % capacity], ..., buf[(front + size - 1) % capacity]
```

Indices wrap around from `capacity - 1` to `0`. **Elements never move inside `buf`**; only `front` and `size` change.

```
capacity = 4, front = 3, size = 3

index     0     1     2     3
buf    [  1  |  2  |  ?  |  0  ]
                           ↑ front

front to back: buf[3], buf[0], buf[1]  ->  0, 1, 2
```

(a) `deque_create`, `deque_free` [10 points]

```
Deque *deque_create(int capacity);
void deque_free(Deque *dq);
```

- `deque_create` returns an empty deque: allocate `buf` with `capacity` slots and set `size` and `front` to `0`. Return `NULL` when `capacity < 1`.
- `deque_free` frees `buf` and the deque. It does nothing when `dq` is `NULL`.

(b) Push, pop and peek [40 points]

```
int deque_is_empty(const Deque *dq);
int deque_is_full(const Deque *dq);
int deque_push_front(Deque *dq, int value);
int deque_push_back(Deque *dq, int value);
int deque_pop_front(Deque *dq, int *out);
int deque_pop_back(Deque *dq, int *out);
int deque_peek_front(const Deque *dq, int *out);
int deque_peek_back(const Deque *dq, int *out);
```

- `deque_is_empty` and `deque_is_full` return `1` or `0`.
- A push fails when the deque is full. A pop or peek fails when it is empty and leaves `*out` untouched.
- A pop writes the element to `*out` and removes it. A peek only writes it to `*out`.

Each operation changes `front` and `size` as follows.

function	front	size	slot it uses
`push_front`	one step left (`0` becomes `capacity - 1`)	$+1$	the new `front`
`pop_front`	one step right (`capacity - 1` becomes `0`)	-1	the old `front`
`push_back`	unchanged	$+1$	`(front + size) % capacity`
`pop_back`	unchanged	-1	`(front + size - 1) % capacity`

For the back operations, the slot is computed with `size` as it was before the call.

Example with `capacity = 4`:

```
call              returns   front   size   front -> back
deque_create(4)               0       0     []
push_back(1)          1       0       1     [1]
push_back(2)          1       0       2     [1, 2]
push_front(0)         1       3       3     [0, 1, 2]
pop_back  -> 2        1       3       2     [0, 1]
pop_front -> 0        1       0       1     [1]
```

(c) `deque_print` [10 points]

```
void deque_print(const Deque *dq);
```

Print the elements from front to back in the format below, followed by a newline.

```
[3, 1, 4]        three elements
[7]              one element
[]               empty deque
```

(d) `round_robin` [40 points]

```
int round_robin(const int *burst, int n, int quantum, int *finish_order);
```

Task `i` needs `burst[i]` units of CPU time. A single CPU runs the tasks in turns:

1. At the start, tasks `0, 1, ..., n-1` wait in a queue in that order.
2. Take the task at the front of the queue and run it for `quantum` units, or for the time it still needs if that is less.
3. If the task is finished, append its index to `finish_order`. Otherwise put it at the back of the queue.
4. Repeat until the queue is empty, then return `n`.

- Every `burst[i]` is at least 1, and `finish_order` has room for `n` entries.
- Return `-1` when `quantum < 1`.
- **Build the queue with your deque from (a) and (b).**
- `burst` is `const`. Keep the remaining times in an array of your own and free it, together with the deque, before returning.

Example: `burst = [5, 2, 4]`, `quantum = 2` ⇒ `finish_order = [1, 2, 0]`, returns `3`

```
step   task   runs   time left   queue (front -> back)   finish_order
start                            0 1 2
1      0      2      3           1 2 0
2      1      2      0 (done)    2 0                     1
3      2      2      2           0 2                     1
4      0      2      1           2 0                     1
5      2      2      0 (done)    0                       1 2
6      0      1      0 (done)                            1 2 0
```

More examples:

```
burst = [3, 1, 2],     quantum = 1   =>  finish_order = [1, 2, 0]
burst = [6, 1, 3, 2],  quantum = 2   =>  finish_order = [1, 3, 2, 0]
burst = [4, 3, 5],     quantum = 10  =>  finish_order = [0, 1, 2]
n = 0                                =>  returns 0
burst = [3, 1],        quantum = 0   =>  returns -1
```

Problem 2. Recently Viewed List (C)

Folder: `exercises/week_06_qe_c/c/impl_recent_list/` **[100 points]**
Files: `recent.h` (do not modify) `solution.c` (write here and submit) `main.c` (self-check driver)

An online shop keeps the list of products a customer viewed recently. Implement it with a **doubly linked list**:

- the most recently viewed product is at the head;
- viewing a product that is already in the list moves it to the head;
- when the list holds more than `capacity` products, the least recently viewed one (the tail) is dropped.

```
typedef struct Item {
    int id;
    struct Item *prev;   /* toward the head: viewed more recently */
    struct Item *next;   /* toward the tail: viewed less recently */
} Item;

typedef struct {
    int capacity;   /* how many products the list remembers */
    int size;       /* how many it holds now */
    Item *head;     /* most recently viewed, NULL when empty */
    Item *tail;     /* least recently viewed, NULL when empty */
} RecentList;
```

```
          head                          tail
           v                             v
 NULL <-- [ 9 ] <-----> [ 2 ] <-----> [ 5 ] --> NULL
          most recent                 least recent
```

After every operation the list must satisfy all of the following. **The grader reads the structs directly and checks them.**

1. `head->prev` and `tail->next` are `NULL`. An empty list has `head` and `tail` both `NULL`.
2. `a->next == b` exactly when `b->prev == a`.
3. Following `next` from `head` ends at `tail`.
4. `size` equals the number of items.

(a) `item_create`, `recent_create`, `recent_free` [10 points]

```
Item *item_create(int id);
RecentList *recent_create(int capacity);
void recent_free(RecentList *list);
```

- `item_create` allocates an item with the given `id` and `prev = next = NULL`.
- `recent_create` returns an empty list (`size` 0, `head` and `tail` `NULL`), or `NULL` when `capacity < 1`.
- `recent_free` frees every item still in the list, then the list itself. It does nothing when `list` is `NULL`.

(b) `recent_push_front` [15 points]

```
void recent_push_front(RecentList *list, Item *item);
```

Link `item`, which is not in any list, in as the new head and add 1 to `size`. Do not look at `capacity`; sub-part (e) handles overflow.

```
before:   head -> [2] <-> [5]             item = [9]
after:    head -> [9] <-> [2] <-> [5]
```

(c) `recent_detach` [20 points]

```
void recent_detach(RecentList *list, Item *item);
```

Unlink `item`, which is in `list`, and subtract 1 from `size`. Set `item->prev` and `item->next` to `NULL`. **Do not free it**: the caller may link it in again.

```
[9, 2, 5]:   detach 2  ->  [9, 5]
             detach 9  ->  [2, 5]
             detach 5  ->  [9, 2]
[7]:         detach 7  ->  []        (head and tail both become NULL)
```

(d) `recent_find` [10 points]

```
Item *recent_find(const RecentList *list, int id);
```

Return a pointer to the item with this `id`, the item inside the list and not a copy, or `NULL` when there is none.

(e) `recent_view` [25 points]

```
void recent_view(RecentList *list, int id);
```

Record that product `id` was viewed.

1. If `id` is already in the list, move that same item to the head. **Do not allocate a new item**; the grader counts calls to `malloc`.
2. Otherwise create an item for `id` and link it in at the head.
3. If step 2 makes `size` exceed `capacity`, remove the tail item and free it.

Example with `capacity = 3`:

```
call                          list (head -> tail)
view(1), view(2), view(3)     [3, 2, 1]
view(2)                       [2, 3, 1]    already in the list: moved to the head
view(4)                       [4, 2, 3]    four items, so the tail 1 is dropped
view(3)                       [3, 4, 2]    moved; nothing is dropped
```

(f) `recent_remove` [10 points]

```
int recent_remove(RecentList *list, int id);
```

Unlink the item with this `id` and free it. Return `1` if it was removed and `0` if `id` is not in the list.

(g) `recent_print` [10 points]

```
void recent_print(const RecentList *list);
```

Print the ids from head to tail in the format below, followed by a newline.

```
5 -> 2 -> 9      list [5, 2, 9]
7                list [7]
(empty)          empty list
```

Problem 3. Graph Search (C)

Folder: `exercises/week_06_qe_c/c/impl_graph_search/` **[100 points]**
Files: `graph.h` (do not modify) `solution.c` (write here and submit) `main.c` (self-check driver)

Store a directed graph as adjacency lists, then implement breadth-first search, depth-first search and cycle detection on it. Vertices are numbered `0` to `n - 1`.

```
typedef struct Edge {
    int to;              /* the vertex this edge points to */
    struct Edge *next;   /* next edge leaving the same vertex */
} Edge;

typedef struct {
    int n;        /* number of vertices */
    Edge **adj;   /* adj[v]: first edge leaving v, NULL when there is none */
} Graph;
```

`adj` is an array of `n` pointers; `adj[v]` is the first node of the linked list of edges leaving `v`. The examples below use **graph G**:

```
graph G                          adjacency lists

  0 --> 1 --> 3 --> 4            adj[0] -> [1] -> [2] -> NULL
  |           ↑                  adj[1] -> [3] -> NULL
  +---> 2 ----+                  adj[2] -> [3] -> NULL
                                 adj[3] -> [4] -> NULL
  5  (no edges)                  adj[4] -> NULL
                                 adj[5] -> NULL
```

G is built by adding the edges `(0,1)`, `(0,2)`, `(1,3)`, `(2,3)`, `(3,4)` in this order. Edges are directed: there is no way from 3 back to 1.

(a) `graph_create`, `graph_add_edge`, `graph_free` [20 points]

```
Graph *graph_create(int n);
int graph_add_edge(Graph *g, int from, int to);
void graph_free(Graph *g);
```

- `graph_create` returns a graph with `n` vertices and no edges: `adj` has `n` entries, all `NULL`. Return `NULL` when `n < 1`.
- `graph_add_edge` appends the edge `from -> to` at the **end** of `adj[from]` and returns `1`. When `from` or `to` is not a vertex it changes nothing and returns `0`. Self-loops are allowed, and adding the same edge twice adds it twice.
- `graph_free` frees every edge, `adj` and the graph. It does nothing when `g` is `NULL`.

Appending at the end matters: the list order is the order used in (b) and (d).

(b) `graph_print` [10 points]

```
void graph_print(const Graph *g);
```

Print one line per vertex, starting from `0`: the vertex, a colon, then each out-neighbour in list order with one space before it. A vertex without edges prints only the vertex and the colon, with no trailing space. Graph G prints:

```
0: 1 2
1: 3
2: 3
3: 4
4:
5:
```

(c) `graph_bfs_distance` [25 points]

```
int graph_bfs_distance(const Graph *g, int start, int *dist);
```

`dist` has room for `n` entries. Set `dist[v]` to the fewest edges on a path from `start` to `v`, or `-1` when `v` cannot be reached; `dist[start]` is `0`. Return the number of reachable vertices, `start` included. When `start` is not a vertex, return `-1` and leave `dist` untouched.

Use breadth-first search with a queue you allocate inside the function, and free it before returning.

```
graph G, start 0   =>  dist = [0, 1, 1, 2, 3, -1],       returns 5
graph G, start 3   =>  dist = [-1, -1, -1, 0, 1, -1],    returns 2
graph G, start 5   =>  dist = [-1, -1, -1, -1, -1, 0],   returns 1
graph G, start 6   =>  dist untouched,                   returns -1
```

(d) `graph_dfs_order` [20 points]

```
int graph_dfs_order(const Graph *g, int start, int *order);
```

Run a depth-first search from `start`. Write the vertices to `order` in the order they are first visited and return how many were visited. Return `-1` when `start` is not a vertex. `order` has room for `n` entries.

When you visit a vertex, go into each of its unvisited out-neighbours **in list order**, and finish exploring one neighbour completely before trying the next.

```
graph G, start 0   =>  order = [0, 1, 3, 4, 2],   returns 5
                       (0 -> 1 -> 3 -> 4, back up to 0, then 2; 3 is already visited)
graph G, start 2   =>  order = [2, 3, 4],         returns 3

G with (0,2) added before (0,1), start 0   =>  order = [0, 2, 3, 4, 1]
```

(e) `graph_has_cycle` [25 points]

```
int graph_has_cycle(const Graph *g);
```

Return `1` when the graph has a directed cycle anywhere, and `0` otherwise. A self-loop counts as a cycle.

```
graph G                              =>  0
0->1, 1->2, 2->0                     =>  1
0->0                                 =>  1
0->1, 0->2, 1->3, 2->3               =>  0   (3 is reached twice, no way back)
0->1, 2->3, 3->2                     =>  1   (cycle not reachable from 0)
```